

Glurg glurg glurg!

Prepared by: Rene Andre B. Jocsing

Published: March 25, 2026

Deadline: May 01, 2026

Your third and final machine problem is to create Yet Another Vampire Survivors Clone (YAVSC) from scratch in Godot. You may use free-to-use art and sound assets, but the implementation and code must come from you.

Learning Objectives

By accomplishing this machine problem, you should be able to:

- Design and implement a data-driven weapon and upgrade system using composition over inheritance
 - Manage large, dynamic entity populations efficiently using object pooling
 - Apply hierarchical and nested FSMs to manage global game flow, enemy behavior, and upgrade selection states
 - Implement an XP and leveling system with signal-driven UI feedback
 - Build a scalable spawn management system with wave-based or time-based difficulty escalation
 - Deepen your understanding of Resource-based configuration, Area2D hit detection, and scene pooling in Godot
 - Demonstrate professional architecture that separates weapon logic, enemy logic, player logic, and game state management into clearly bounded systems
 - Practice writing systems capable of supporting emergent gameplay from the combination of independent components
-

Why Vampire Survivors?

Vampire Survivors appears deceptively trivial at first glance. The player moves. Weapons fire automatically. Enemies flood the screen. You pick upgrades every few levels. The loop repeats until you die or the timer runs out.

Underneath this apparent simplicity is one of the most architecturally demanding game designs you will encounter as a developer. Vampire Survivors is, at its core, a **stress test for your systems thinking**. The moment you try to build it, you will encounter every major scalability problem in game engineering simultaneously.

How do you represent a weapon system that supports hundreds of combinations of base weapons and their evolved forms without drowning in subclasses? How do you keep three hundred enemies alive on screen at 60 frames per second without incinerating the CPU? How do you design an upgrade selection screen that works correctly regardless of which weapons and passives the player has already collected? How do you tune a difficulty curve that scales smoothly across thirty minutes of play without rewriting half the game for each change?

These are not academic questions. They are the exact problems that ship real games or sink them. Machine Problem 1 taught you the basics of a complete game loop. Machine Problem 2 taught you emergent complexity through multi-agent interaction. Machine Problem 3 teaches you **scale**: what it means to architect a system that stays clean when the number of entities, weapon types, upgrade combinations, and concurrent effects multiplies beyond what you can hold in your head (and roguelikes are popular in the market today).

By the time you are done, you will have built—from scratch—the skeleton of a system that studios with full engineering teams spend months designing. That is a high bar, but this is the final machine problem for this semester.

The Original Mechanics

The Player Character

The player controls a single character that moves freely in all eight directions across an unbounded (or very large, wrapping) arena. Movement is the player's primary agency—their only direct interaction with the world. It must feel responsive and fluid. Use `CharacterBody2D` with `move_and_slide()`; inertia and drag are design decisions but may improve game feel considerably.

The character has the following core attributes, all of which must be externalized as configurable data (exported variables or a Resource configuration), not inline literals:

- **Max HP:** Total health points; reaching zero triggers the death state
- **Movement Speed:** Base walk speed, upgradeable via passives
- **XP Magnet Radius:** The radius within which XP gems are automatically pulled toward the player
- **Armor:** Flat damage reduction applied before HP loss

When the player is hit, a brief invulnerability window (i-frames) must be applied to prevent multiple hits from the same collision frame. This is not optional—without it, the game becomes unplayable.

The Weapon System

This is the architectural heart of the machine problem. Every weapon fires automatically on a per-weapon timer. The player does not aim or trigger weapons manually. The design challenge is implementing a weapon system that is:

- **Data-driven:** Each weapon's behavior is defined by a configuration resource, not hard-coded logic
- **Composable:** Weapons can be upgraded independently; passive items modify weapon stats without touching weapon code
- **Extensible:** Adding a new weapon requires writing a new scene and a new resource, not modifying any existing weapon or manager class

You must implement **at least three distinct weapons** with meaningfully different attack patterns. Suggested archetypes:

- **Projectile weapon:** Fires one or more homing or directional bullets on a cooldown (e.g., a wand that shoots at the nearest enemy)
- **Area weapon:** Deals damage in a radius around the player on a cooldown (e.g., a garlic aura that pulses every second)
- **Orbiting weapon:** Places rotating bodies around the player that deal damage on contact (e.g., revolving knives or axes)

Architecture Requirement: Composition Over Inheritance

Your weapon system must use the Component pattern or a data-driven Resource pattern to define weapon behavior. A weapon class hierarchy where `HomingBulletWeapon` and `AreaPulseWeapon` inherit from a base `Weapon` class is acceptable only if the base class is thin and behavior is delegated to composable data resources.

What is **not** acceptable: a monolithic `Weapon.gd` with fifty exported booleans and a massive `_process()` function that uses `if` chains to select behavior. This is the God Object anti-pattern. It will not survive the addition of a fourth weapon type. Design accordingly.

The XP and Leveling System

Enemies drop XP gems on death. When the player collides with (or magnetically collects) a gem, XP is added to a running total. When the total crosses the threshold for the current level, the player levels up.

On level-up, the game must **pause** and present the player with an **upgrade selection screen**: a choice of three (or more) randomly drawn options from a pool of available weapon upgrades and passive item improvements. The player selects one, the game unpauses, and the chosen upgrade is applied immediately and permanently for the rest of the run.

The upgrade pool must be dynamic: weapons the player already owns offer level-up upgrades; weapons they do not own may offer first-acquisition options; passive items always offer stat improvements up to their own caps. The pool must never offer the player a choice they cannot legally take (e.g., a weapon already at max level should not appear in the pool unless you intend for max-level weapons to offer evolution options).

XP thresholds must scale with level. The formula is your design choice, but it must be externalized as a configurable curve, not a magic number.

Enemy Spawning and Population Management

Enemies spawn continuously from outside the player's view and move toward the player. The difficulty escalates over time: the quantity, variety, and HP of enemies increase as the run clock advances.

A dedicated **spawn manager** node or autoload is responsible for the entire population. Its parameters—spawn rate, active enemy cap, enemy type distribution, difficulty curve—must all be exported or resource-configured. Hard-coded spawn logic will be treated as a spaghetti code violation for grading purposes.

Object pooling is required for this machine problem. Vampire Survivors routinely maintains hundreds of active entities. Instantiating and freeing nodes continuously at that scale causes visible frame drops. You must implement a pool for at minimum the most frequently spawned enemy type and for projectiles. This is not optional.

Object Pooling in Godot

A minimal pool maintains a list of pre-instantiated inactive nodes. To acquire an entity from the pool, call a method that activates and returns an inactive instance. To release an entity back, call a method that deactivates it and returns it to the available list. Neither `instantiate()` nor `queue_free()` should be called during gameplay for pooled types—only during pool initialization and teardown.

Enemy Behavior

At a minimum, enemies must move toward the player's current position. This is sufficient for basic grunts—and in *Vampire Survivors*, it is exactly what most enemies do. However, your implementation must be FSM-driven per entity. A flat `_process()` function that unconditionally moves toward the player does not constitute an FSM and will be graded accordingly.

A minimum two-state FSM (e.g., **Move** and **Dead**) is required. A three-state FSM (e.g., **Spawn**, **Move**, **Dead**) is recommended. More complex behaviors (knockback, charge attacks, patrol-then-pursue) are valid additions for your mechanical twist. Look into pathfinding algorithms and artificial intelligence in games to enhance your enemies.

Damage, Death, and Feedback

When a weapon hits an enemy, the enemy takes damage. When HP reaches zero, the enemy plays a death effect, drops an XP gem, and returns to the pool. Floating damage numbers are optional but strongly recommended for game feel.

When an enemy hits the player, the player takes damage minus armor, i-frames activate, and a visual feedback effect (flash, screen shake, or color modulation) must play. If the player reaches 0 HP, the game transitions to the Game Over state.

The Run Timer and Win Condition

Each run lasts a fixed duration (commonly 30 minutes in *Vampire Survivors*, but you should start with 5–10 minutes for testing). A visible countdown or elapsed timer must be displayed. When the timer reaches the end, a **final boss wave** triggers—a single, dramatically stronger enemy that the player must defeat to win. Defeating it triggers a Win State. The player dying before the timer ends triggers a Game Over State.

Your game does not need to match *Vampire Survivors*' full 30-minute structure, but it must have a defined arc: escalating difficulty, a climax, and a resolution.

Game States

Your game requires at minimum four states managed by a top-level FSM:

- **Menu State:** Pre-game; displays title, character/weapon selection if applicable
- **Playing State:** Active run with movement, auto-attacks, spawning, and XP gain
- **Upgrade State:** Paused mid-run; player selects a level-up reward
- **Game Over / Win State:** Run ended; displays final stats and restart option

The Upgrade State is a **sub-state within the Playing State**—the run timer must pause, enemy movement must freeze, weapons must stop firing, and input must be routed to the upgrade UI exclusively. Resuming from Upgrade State must restore the game cleanly with no residual effects.

Your Original Mechanics

What Makes a Valid Twist?

Your clone must include a **unique mechanical twist (or several)** that meaningfully changes gameplay. This is NOT cosmetic.

Valid Examples:

- A risk/reward system where the player can voluntarily take damage to fuel a powerful ability, making survivability a resource
- A weapon evolution system driven by enemy type: defeating enough of a specific enemy type unlocks a weapon's evolved form regardless of passive items
- A "corruption" mechanic where the player's weapons deal more damage the lower their HP, creating a high-risk offensive build
- An asymmetric co-op mode where one player moves and one player controls weapon aim or selects upgrades
- A prestige/ascension system where surviving a shorter time with a strong handicap earns permanent cross-run bonuses stored to a save file

Invalid Examples:

- Reskinned enemies or weapons with no behavioral difference
- A different visual theme or camera angle without mechanical consequence
- Adding background music or ambient effects only
- Changing UI colors, fonts, or icon styles

Ask yourself: Does this change *how* the player thinks about survival, build construction, or resource management—or does it only change *what they look at*? Will I get struck for copyright infringement if I publish this on Steam or Google Play?

You may deviate from the Technical Requirements section if and only if: it benefits or is crucial to your original mechanics, or is architecturally sound with justification!

Technical Requirements

Architecture

- **Strong typing:** All function signatures and class members must be type-annotated (whether explicitly through Kotlin-style typehints or implicitly through Python-style walrus operators)
- **State machines:** Enum-based FSM for global game state; separate FSM per enemy entity for behavior states; Upgrade State correctly pauses and resumes Playing State
- **Scene composition:** Separate scenes for Player, each Weapon type, Enemy, XP Gem, SpawnManager, UpgradeUI, HUD, and Main
- **Signal-driven:** Use signals for decoupled communication (player leveled up, enemy died, upgrade selected, run ended, HP changed)
- **Data-driven configuration:** Weapon stats, enemy stats, XP curves, and spawn parameters must be `@export` variables or `Resource` configurations, not inline literals

- **Object pooling:** Pooling required for projectiles and at least one enemy type
- **Separation of concerns:** Weapon logic, enemy AI, XP/leveling, spawning, and upgrade selection live in distinct scripts and/or nodes; no cross-domain logic bleeding into a single monolithic script

Common Pitfalls

Spaghetti code will cap your grade at 3.00 regardless of gameplay quality. Architecture is heavily weighted in grading. A `Main.gd` that manages spawning, weapon firing, XP tracking, upgrade pools, and enemy movement simultaneously is not acceptable—even if the game runs flawlessly.

Input Handling

The player character is the only node that accepts direct player input during the Playing State. All other input (upgrade selection, restart, pause) must be routed through the appropriate state and its designated UI nodes. During the Upgrade State, movement and weapon input must be suppressed. During the Game Over/Win State, only the restart or menu input should be active.

Collision Detection

Implement collision detection for all of the following interactions:

- Weapon projectiles or areas hitting enemies
- Enemy bodies hitting the player (damage application with i-frame check)
- XP gems entering the player's magnet radius (auto-collect trigger)
- XP gems directly contacting the player (collect trigger, if magnet is not active)
- Projectiles hitting the arena boundary or exceeding their range (return to pool)

Use `Area2D` nodes with collision shapes for all detection. Configure physics layers and masks deliberately—weapon areas must not detect each other, enemy bodies must not push other enemies out of position, and XP gems must not block player movement.

Entity Behavior (Enemy AI)

Each enemy type must operate under its own FSM with at least two states. Transitions must be triggered by game events (spawn complete, death triggered), not by raw flags. Detection and proximity checks should use `Area2D` overlap signals where applicable rather than per-frame distance polling for every enemy. Performance matters at scale.

Visual and Audio Feedback

At minimum, your game should include:

- Distinct sprites for the player, each enemy type, each weapon/projectile type, and XP gems
- Visual scaling or color modulation for different enemy HP tiers or levels
- I-frame flash effect when the player is hit
- Death effect for enemies (particle burst or animation, then return to pool)
- Level-up effect (screen flash, sound cue) on XP threshold crossing

- A HUD displaying: current HP, current XP progress toward next level, current level, elapsed run time, and active weapons/passives
- Sound effects for: weapon firing, enemy death, player hit, level-up, upgrade selection
- Background music with at minimum two tracks (menu and gameplay)

Free assets available at: [OpenGameArt.org](https://opengameart.org/), itch.io, [Kenney.nl](https://kenney.nl/). Ensure assets have appropriate licenses for educational use. Credit all assets in CREDITS.md.

Deliverables

1. *The Game*

An original playable Godot 4 project based on Vampire Survivors, implementing all core mechanics and technical requirements. The game must be stable for at least one complete run—no crashes, no orphaned nodes, no memory leaks detectable across a full session. The core loop must work flawlessly: survive, collect XP, level up, pick upgrades, survive longer, reach the boss, win or die.

2. *GitHub Repository*

Your complete Godot project must be hosted in a GitHub repository with clean commit history. Include:

- All Godot project files and scenes
- All assets with proper attribution in CREDITS.md
- README.md containing:
 - Game description and mechanical twist explanation
 - Controls documentation
 - Known issues or limitations
 - Asset credits and licenses
- Built game binary zipped in GitHub Releases section

Invite the instructor as a collaborator. Use semantic commit messages following the conventions from the syllabus.

3. *Presentation (15 minutes)*

Prepare a jam-style presentation covering:

- Live gameplay demonstration
- Architecture decisions and design patterns used—with special attention to the weapon and upgrade systems
- Explanation of your mechanical twist
- Challenges faced and solutions
- Q&A from classmates and instructor

Class will vote on: Best Architecture, Best Twist, Best Polish. Winners receive +5% extra credit per category. This can stack up to +15%!

Weekly Progress Reports

Weekly progress reports during consultation hours are **strongly encouraged**. Groups that attend receive:

- Ongoing feedback on architecture—especially on the weapon composition and object pool designs before they calcify into technical debt
- Early detection of performance issues (three hundred enemies is not something you can stress-test for the first time the night before the deadline)
- Code review and refactoring guidance
- Up to +5% extra credit for consistent attendance

Groups that skip progress reports accept the risk of discovering major architectural flaws during final presentations when fixes are costly or impossible, as well as forfeit the extra credit.

Grading Philosophy

This is not a competition to build the most content-rich Vampire Survivors clone. Vampire Survivors itself shipped with a fraction of its final content at launch. The goal is to demonstrate that you can design and implement a system of sufficient architectural maturity that adding a fourth weapon, a fifth passive, or a new enemy type requires only writing new data and new scenes—not rewriting the systems that already work.

A three-weapon, three-enemy, well-architected game will score better than a ten-weapon, ten-enemy, poorly-architected one.

Focus on:

- Clean, composable weapon and upgrade architecture
- Correct and performant entity management (pooling matters)
- Meaningful and coherently integrated mechanical twist
- Professional commit history and documentation
- Excellent game “feel”—auto-attacking weapons should feel satisfying even before the player upgrades them

Extra features (evolved weapons, achievement systems, meta-progression) are welcome after the foundation is solid, but will not compensate for poor architecture.

Pair Programming Expectations

- Both members must contribute to code and design
- Use Git branches, pull requests, code review
- Maintain clear communication and regular check-ins
- Document design notes in README.md
- Both members must speak during presentation

Individual accountability assessed through:

- GitHub commit history

- Presentation participation
- Consultation session involvement
- Peer evaluation (if issues arise)

See the syllabus for more details regarding the pair programming format.

Academic Honesty

The usage of Large Language Models (e.g., ChatGPT, Claude, Deepseek) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course and harsh disciplinary action.

You are expected to write your own code and understand every line you submit. During presentations and consultations, you must be able to explain and defend your architectural decisions and implementation choices.

Important Dates

Weekly Progress Reports

Progress reports are conducted during office hours on a first-come, first-served basis. Attendance is optional but **strongly encouraged**, and can earn up to +5% extra credit for the machine problem.

- **Week 1:** April 7, 2026
- **Week 2:** April 14, 2026
- **Week 3:** April 21, 2026
- **Week 4:** April 28, 2026

Submission of GitHub Repository

The GitHub repository for the machine problem must be submitted (with completed code and release build) on **May 1, 2026**. To submit, simply invite the instructor as a collaborator. Late submissions will be penalized with a -25% deduction. Commits dated after the deadline will not be considered during evaluation.

Final Presentation

Machine problem presentations will be conducted in a jam-style format during regular class hours. Both team members must be present and participate.

- **Presentation Date:** Friday, May 5, 2026
- **Duration:** 15 minutes per team (demo + Q&A)
- **Format:** Live gameplay demo, architecture showcase and explanation, voting

Note: Machine problems that are functional but exhibit significant architectural issues (e.g., spaghetti code, no FSM, hard-coded values, tight coupling, no object pooling for high-frequency entities) will receive a maximum grade of 3.00 (60/100) regardless of gameplay quality or presentation performance. Professional game development prioritizes maintainable, scalable systems over short-term functionality.

Grading Rubric

**Score may vary individually for component.*

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Architecture & Design (25%)	Clean separation of concerns, proper global and per-entity FSM implementation, composable weapon/upgrade system, object pooling for high-frequency entities, signal-driven architecture, data-driven configuration	Minor architectural issues, most patterns implemented correctly, pooling present, consistent structure	Significant architectural problems, pooling absent or trivial, inconsistent pattern usage, weapon logic bleeds into unrelated scripts	Spaghetti code, no clear architecture, monolithic scripts, hard-coded weapon stats, no pooling, violates OOP principles
Mechanical Twist (20%)	Creative, meaningful gameplay innovation that reshapes build strategy or moment-to-moment decision-making; seamlessly integrated into core loop without breaking existing mechanics	Valid twist that changes gameplay meaningfully, decent integration	Minimal twist or poorly integrated, feels bolted on rather than designed into the systems	Cosmetic only, missing, or breaks core mechanics

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Code Quality (10%)	Strongly typed signatures and members, excellent naming conventions, well-documented using Godot conventions, exported parameters for all tunable values	Mostly typed, good naming, adequate documentation, some magic numbers present	Inconsistent typing/naming, sparse documentation, configuration buried in logic	Poor typing, unclear code, missing documentation, hard-coded values throughout
Individual Contributions* (20%)	Equitable commits with semantic messages, clear authorship, professional Git workflow (branches, PRs)	Mostly balanced contributions, adequate commit messages, basic Git usage	Uneven contributions visible in history, poor commit messages	One member dominates commits, nonsemantic messages, commit noise
Presentation* (25%)	Professional demo with smooth gameplay showing weapon variety and upgrade flow, clear architecture explanation with emphasis on composability, both members articulate design decisions, handles Q&A confidently and pitches game well	Good demo, adequate explanations of implementation, both members participate meaningfully, pitch is good	Rough demo with bugs, unclear explanations, uneven participation, struggles with questions about weapon system design, pitch is questionable	Unprepared, can't explain code or architecture, one member dominates, uncertain responses, pitch needs revision